

第 11 章作业

1.在数据库中为什么要并发控制？并发控制技术能保证事务的哪些特征？

答：数据库是共享资源，通常多个事务同时运行。当多个事务并发地存取数据库时就会产生同时读取和/或修改同一数据的情况。若对并发操作不加控制就可能会存取和存储不正确的数据，破坏数据库的一致性。所以数据库管理系统必须提供并发控制机制。

并发控制能保证事务的一致性和隔离性。

2.并发操作可能会产生哪几类数据不一致？用什么方法能避免各种不一致的情况？

答：三类：

(1) 丢失修改。两个事务 T_1 和 T_2 读入同一数据并修改， T_2 提交的结果破坏了 T_1 提交的结果，导致了 T_1 的修改被丢弃。

(2) 不可重复读。不可重复读是指事务 T_1 读入同一数据后，事务 T_2 执行更新操作，使 T_1 无法再现前一次读取结果。具体地讲，不可重复读包括三种情况：

① 事务 T_1 读取某一数据后，事务 T_2 对其进行了修改，当事务 T_1 再次读该数据时，得到与前一次不同的值。

② 事物 T_1 按一定条件从数据库中读取了某些数据记录后，事务 T_2 删除了其中部分记录，当 T_1 再次按相同条件读取数据时，发现某些记录消失了。

③ 事务 T_1 按一定条件从数据库中读取某些数据库记录后，事物 T_2 插入了一些记录，当 T_1 再次按相同条件读取数据时，发现多了一些记录。

后两种不可重复读有时也称为幻影现象。

(3) 读“脏”数据。读“脏”数据是指事务 T_1 修改某一数据，并将其写回磁盘，事务 T_2 读取同一数据后， T_1 由于某种原因被撤销，这时 T_1 已修改过的数据恢复原值， T_2 读到的数

据就与数据库中的数据不一致，则 T_2 独到的数据就为“脏”数据，即不正确的数据。

避免不一致性的方法就是并发控制。常用的并发控制技术包括封锁方法、时间戳方法、乐观控制方法和多版本并发控制方法等。

3.什么是封锁？基本的封锁类型有几种？试述他们的含义。

答：封锁就是事务 T 在对某个数据对象例如表、记录等操作之前，先向系统发出请求，对其加锁。加锁后事务 T 就对该数据对象有了一定的控制。在事务 T 释放它的锁之前，其他的事务不能更新或读取此数据对象。

基本的封锁类型有两种：排他锁和共享锁。

排他锁又称为写锁：若事务 T 对数据对象 A 加上 X 锁，则只允许 T 读取和修改 A ，其他任何事物都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁。这就保证了其他事务在 T 释放 A 上的锁之前不能再读取和修改 A 。

共享锁又称为读锁。若事务 T 读数据对象 A 加上 S 锁，则事务 T 可以读 A 但不能修改 A ，其他事物只能对 A 加 S 锁，而不能加 X 锁，直到 T 释放 A 上的 S 锁。这就保证了其他事务可以读 A ，但在 T 释放 A 上的 S 锁之前不能对 A 做任何修改。

4.如何用封锁机制保证数据的一致性？

答：按照一定的封锁协议，运用 X 锁和 S 锁对数据对象加锁实现。这些封锁协议包括：

(1) 一级封锁协议：事物 T 在修改数据 R 之前必须先对其加 X 锁，直到事务结束才释放。事物结束。

(2) 二级封锁协议：在一级封锁协议的基础上增加事务 T 在读取数据 R 之前必须先对其加 S 锁，读完后即可释放 S 锁。

(3) 三级封锁协议：在一级封锁协议的基础上增加事务 T 在读取数据 R 之前必须先对其加 S 锁，直到事务结束才释放。

5.什么是活锁？试述活锁的产生原因和解决方法。

答：如果事务 T_1 封锁了数据 R ，事物 T_2 又请求封锁 R ，于是 T_2 等待； T_3 也请求封锁 R ，当 T_1 释放了 R 上的封锁之后系统首先批准了 T_3 的请求， T_2 仍然等待；然后 T_4 又请求封锁 R ，当 T_3 释放了 R 上的封锁之后系统又批准了 T_4 的请求..... T_2 有可能永远等待，这就是活锁的情形。

活锁的产生原因：当一系列封锁不能按照其先后顺序执行时，就可能导致一些事务无限期等待某个封锁，从而导致活锁。

避免活锁的简单方法是采用先来先服务的策略。当多个事务请求封锁同一数据对象时，封锁子系统按请求封锁的先后次序对事务排队，数据对象上的锁一旦释放就批准申请队列中第一个事务获得锁。

6.什么是死锁？请给出预防死锁的若干方法。

答：如果事务 T_1 封锁了数据 R_1 ， T_2 封锁了数据 R_2 ，然后 T_1 又请求封锁 R_2 ，因 T_2 已封锁了 R_2 ，于是 T_1 等待 T_2 释放 R_2 上的锁；接着 T_2 又申请封锁 R_1 ，因 T_1 已封锁了 R_1 ， T_2 也只能等待 T_1 释放 R_1 上的锁。这样就出现了 T_1 在等待 T_2 ，而 T_2 又在等待 T_1 的局面， T_1 和 T_2 两个事务永远不能结束，形成死锁。

预防死锁的方法：

(1) 一次封锁法。要求每个事务必须一次奖所有要使用的数据全部加锁，否则就不能继续执行。

(2) 顺序封锁法。余弦对数据对象规定一个封锁顺序，所有事务都按这个顺序实行封锁。

7.请给出检测死锁发生的一种方法，当发生死锁后如何解除死锁？

答：超时法：如果一个事务的等待时间超过了规定的时限，就认为发生了死锁。

解除死锁方法：选择一个处理死锁代价最小的事务，将其撤销，释放此事务持有的所有锁，

	等待	Xlock A
	等待	$A \leftarrow A ** 2$ $W(A)=4$ Unlock A
	$R(A)=4$ Unlock A Xlock A $A \leftarrow A * 2$ $W(A)=8$ Unlock A	

最后结果 A 为 8，为串行化的调度。

(3) 请给出一个非串行化的调度，并给出执行结果。

答：

T ₁	T ₂	T ₃
Slock A $R(A)=0$ Unlock A Xlock A 等待 $A \leftarrow A + 2$ $W(A)=2$ Unlock A	 Slock A $R(A)=2$ Xlock A $A \leftarrow A * 2$ $W(A)=4$ Unlock A Unlock A	 Slock A $R(A)=0$ Unlock A Xlock A $A \leftarrow A ** 2$ $W(A)=0$ Unlock A

最后结果 A 为 0，为非串行化的调度。

(4) 若这三个事务都遵守两段锁协议，请给出一个不产生死锁的可串行调度。

T ₁	T ₂	T ₃
----------------	----------------	----------------

Slock A $R(A)=0$ Xlock A $A \leftarrow A+2$ $W(A)=2$ Unlock A Unlock A	Slock A $R(A)=2$ Xlock A 等待 $A \leftarrow A*2$ $W(A)=4$ Unlock A Unlock A	Slock A $R(A)=4$ 等待 Xlock A $A \leftarrow A**2$ $W(A)=16$ Unlock A Unlock A
--	--	--

(5) 若这三个事务都遵守两段锁协议，请给出一个产生死锁的调度。

T ₁	T ₂	T ₃
Slock A $R(A)=0$ Xlock A 等待	Slock A $R(A)=0$ Xlock A 等待	Slock A $R(A)=0$ Xlock A 等待

10.今有三个事务的一个调度 $r_3(B)r_1(A)w_3(B)r_2(B)r_2(A)w_2(B)r_1(B)w_1(A)$, 该调度是冲突可串行化的调度吗? 为什么?

答:交换 $r_1(A)$ 和 $w_3(B)$,得到 $r_3(B)w_3(B)r_1(A)r_2(B)r_2(A)w_2(B)r_1(B)w_1(A)$,再交换 $r_1(A)$ 和 $r_2(B)r_2(A)w_2(B)$, 得到 $r_3(B)w_3(B)r_2(B)r_2(A)w_2(B)r_1(A)r_1(B)w_1(A)$, 该调度是串行的, 因此 $r_3(B)r_1(A)w_3(B)r_2(B)r_2(A)w_2(B)r_1(B)w_1(A)$ 是可串行化的调度。

11.试证明若并发事务遵守两段锁协议, 则对这些事务的并发调度是可串行化的。

证明: 首先以两个并发事务 T_1 和 T_2 为例进行证明, 对于多个并发事务的情形可以类推。根据可串行化定义可知, 事务不可串行化只可能发生在以下两种情况:

- (1) 事务 T_1 写某个数据对象 A , T_2 读或写 A ;
- (2) 事务 T_1 读或写某个数据对象 A , T_2 写 A 。

下面称 A 为潜在冲突对象。

设 T_1 和 T_2 访问的潜在冲突的公共对象为 $|A_1, A_2, \dots, A_n|$ 。

假设这组潜在冲突中 $X=|A_1, A_2, \dots, A_i|$ 均符合情况 (1), $Y=|A_{i+1}, A_{i+2}, \dots, A_n|$ 均符合情况 (2)。

$\forall x \in X, T_1$ 需要 $Xlock\ x$ (a)

T_2 需要 $Slock\ x$ 或 $Xlock\ x$ (b)

情况 (1) 如果操作 (a) 先执行, 则 T_1 获得锁, T_2 等待。由于遵守两段锁协议, T_1 在成功获得 X 和 Y 中全部对象及非潜在冲突对象的锁后, 才会释放锁。这时如果 $\exists x \in X$ 或 Y , T_2 已获得 w 的锁, 则出现死锁; 否则, T_1 在对 X 、 Y 中对象全部处理完毕后, T_2 才能执行。

这相当于按 T_1 、 T_2 的顺序串行执行。根据可串行化定义, T_1 和 T_2 的调度是可串行化的。

12.举例说明对并发事务的一个调度是可串行化的, 而这些并发事务不一定遵守两段锁协议。

答: $T_1: A=B+1; T_2: B=A+1;$

T ₁	T ₂
Slock B R(B)=2 Unlock B Xlock A W(A)=B+1 Unlock A	 Slock A 等待 等待 Slock A R(A)=3 Unlock A Xlock B W(B)=A+1 Unlock B

13.考虑如下的调度，说明这些调度集合之间的包含关系。

- (1) 正确的调度。
- (2) 可串行化的调度。
- (3) 遵循两阶段封锁（2PL）的调度。
- (4) 串行调度。

答：遵循两阶段封锁（2PL）的调度 $\subset$ 正确的调度=可串行化的调度；串行调度 $\subset$ 正确的调度。

14.考虑 T₁和 T₂两个事务。

T₁: R(A); R(B); B=A+B; W(B)

T₂: R(B); R(A); A=A+B; W(A)

- (1) 改写 T₁和 T₂，增加加锁操作和解锁操作，并要求遵循两阶段封锁协议。

答：T₁: Slock A; R(A); Xlock B; R(B); W(B)=A+B; Unlock A; Unlock B;

T₂: Slock B; R(B); Xlock A; R(A); W(A)=A+B; Unlock B; Unlock A。

- (2) 说明 T₁和 T₂的执行是否会引起死锁，给出 T₁和 T₂的一个调度并说明之。

答：可能会产生死锁。

T ₁	T ₂
Slock A R(A)	
	Slock B R(B)
Xlock B	
等待	Xlock A
	等待

15.为什么要引进意向锁？意向锁的含义是什么？

答：为了提高封锁子系统的效率。在多粒度封锁方法中，一个数据对象能以两种方式加锁——显式封锁和隐式封锁。因此系统在对某一数据对象加锁时，不仅要检查该数据对象上是否有封锁与之冲突，还要检查其所有上级结点和所有下级结点，看申请的封锁是否与这些节点上的封锁冲突。显然这样的检查方法效率很低，因此引进意向锁。

含义：对任一结点加锁时，必须先对它的上层结点加意向锁。引进意向锁后，系统对某一数据对象加锁时不必逐个检查与下一级结点的封锁冲突了。

16.试述常用的意向锁：IS 锁、IX 锁、SIX 锁，给出这些锁的相容矩阵。

答：IS 锁：如果对一个数据对象加 IS 锁，表示它的后裔结点拟加 S 锁。例如，要对某个元组加 S 锁，则要首先对关系和数据库加 IS 锁。

IX 锁：如果对一个数据对象加 IX 锁，便是它的后裔结点拟加 X 锁。例如，要对某个元组加 X 锁，则要首先对关系和数据库加 IX 锁。

SIX 锁：如果对一个数据对象加 SIX 锁，表示对它加 S 锁，再加 IX 锁，即 SIX=S+IX。

相容矩阵：

T ₁ \T ₂	S	X	IS	IX	SIX	-
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y

IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
-	Y	Y	Y	Y	Y	Y

